LangGraph

Dokumentacija: https://docs.langchain.com/oss/python/langgraph/overview

GitHub: https://github.com/langchain-ai/langgraph

LangGraph omogućava organizaciju agenata i drugih komponenti kroz graf, pri čemu čvorovi predstavljaju aktivnosti, a veze definišu tok izvršavanja.

- Omogucava da se vise agenata organizuje u jedan kontrolisani tok izvrsavanja, sa zajednickim podacima, grananjem, paralelnim izvršavanjem i povratnom spregom.
- Dinamicku registraciju proizvoljnih eksternih agenata bih morala dodatno da projektujem
- Ne moram sama da programiram logiku prelaska izmedju odredjenih aktivnosti, vec ih predstavim kao graf izvrsavanja

- **State** - struktura koja čuva podatke potrebne tokom izvršavanja grafa. (Tema kursa: Mašinsko učenje.)
- State je jedan od načina da izlaz prethodne faze postane ulaz naredne faze.

- **Node** (cvor) – funkcija koja obavlja odredjeni posao (LLM agent koji generiše tekst, funkcija koja generiše PDF)
- Čvor može biti obična funkcija, složen agent ili čak čitav podgraf.

- **Edge** (veza) - odredjuje koji cvor se izvrsava nakon prethodnog
- Postoje dve važne vrste veza:
- Obične veze: unapred je poznato koji korak sledi.
- Uslovne veze: sledeći korak zavisi od rezultata prethodnog koraka.

- Čvor može biti obična funkcija, složen agent ili čak čitav podgraf.
- Kako se povezuje vise agenata - Pomoću `StateGraph` objekta i metoda `add_node()` i `add_edge()`.

- Kako se podaci prenose izmedju agenata - Preko zajedničkog stanja - Agenti ne moraju direktno da pozivaju jedni druge. LangGraph upravlja izvršavanjem, a stanje prenosi potrebne podatke.
- U složenijem sistemu možeš koristiti različite šeme podataka i podgrafove. Kod paralelnog izvršavanja posebno je važno definisati kako se istovremene izmene istog polja kombinuju. LangGraph za to koristi reducers.
- Kako se odredjuje redosled izvrsavanja i vracanje na prethodni korak - Redosled određuju veze između čvorova

- **Podgrafovi** - LangGraph podržava organizaciju složenog sistema u manje grafove koji se mogu koristiti kao delovi većeg grafa.
- Svaka faza može da ima sopstveni podgraf sa agentima, vezama i pravilima izvršavanja

- **Paralelno izvrsavanje i sekvencijalno** – podrzava oba pristupa
- Nezavisni čvorovi mogu se izvršavati paralelno, dok zavisni čekaju potrebne rezultate.

- **Persistence** – cuvanje stanja
- Cuvanje stanja izvrsavanja pomocu checkpointer-a. To znači da sistem može da sačuva napredak i nastavi izvršavanje nakon prekida
- Postoji i koncept Store, koji omogućava čuvanje podataka izvan pojedinačnog izvršavanja grafa.

State – trenutno stanje procesa.

Checkpointer – čuva verzije stanja tokom izvršavanja.

Store – omogućava dugotrajnije čuvanje podataka dostupnih kroz različita izvršavanja.

- **Human-in-the-loop – ukljucivanje coveka**
- omogućava da se izvršavanje privremeno zaustavi i sačeka ljudska odluka.

- **Integracija eksternih agenata** - LangGraph čvor može da izvršava Python kod koji poziva eksterni servis.
- LangGraph sam po sebi ne obezbeđuje automatski registar proizvoljnih udaljenih agenata po njihovim sposobnostima
- Za to bih mogla da napravim sopstveni sloj koji cuva identifikator, naziv agenta, fazu kojo pripada, zadatke koje podrzava...
- Engine bi na osnovu tih informacija birao i pozivao odgovarajućeg agenta.

Vise agenata u jednom sistemu – Cvorovi i grafovi

Cetiri faze nastavnog procesa – Glavni graf i podgrafovi

Razmena podataka – State

Redosled izvrsavanja – Edges

Paralelni agenti – Paralelne grane

Povratna sprega – Conditional edges i ciklusi

Cuvanje rezultata – Persistence

Eksterni agenti – Pozivanje API-ja iz cvorova, uz sopstveni registar

Dinamicka registracija agenata – Potreban dodatni projektni mehanizam

CrewAI

Dokumentacija: https://docs.crewai.com/

GitHub: https://github.com/crewAIInc/crewAI

CrewAI omogućava definisanje specijalizovanih agenata, njihovih zadataka i koordinaciju rada kroz timove i tokove izvršavanja.

- zasnovan na specijalizovanim agentima koji imaju jasno definisane uloge, ciljeve i zadatke
- razmišljaš o tome koji agent radi koji zadatak, kako agenti formiraju tim i ko koordinira njihov rad
- Osnovna ideja je da se veliki problem podeli na manje zadatke, a da se svaki zadatak dodeli agentu specijalizovanom za određenu oblast.
- Svaki agent ima svoju odgovornost, a CrewAI omogućava da ih organizuješ u tim koji zajednički izvršava zadatke.

- **`Agent`** predstavlja AI agenta koji ima određenu ulogu, cilj i opis iskustva ili konteksta (backstory).
- **`Task`** opisuje posao koji treba izvršiti.
- **`Crew`** je grupa agenata koji zajednički rade na određenom cilju.
- **`Flow`** omogućava definisanje šireg toka izvršavanja, povezivanje aktivnosti, grananje i upravljanje stanjem.

- Kako se definise jedan agent – pomocu klase Agent, gde je modelovan kao AI izvrsilac sa ulogom i ciljem
- Kako se povezuje vise agenata – kroz Crew, tako što definišeš listu agenata, njihove zadatke i proces izvršavanja.
- **Sequential** – sekvencijalno izvršavanje (Zadaci se izvršavaju unapred definisanim redosledom)
- **Hierarchical** – hijerarhijsko izvršavanje
- Kako se podaci prenose izmedju agenata
  1. preko rezultata zadataka (`Task context`)
  Agent za nastavne ciljeve generiše listu ciljeva, a agent za lekcije koristi taj rezultat
  2. preko zajedničkog stanja Flow-a
  Flow može da čuva podatke kao što su kurikulum, lekcije, rezultati simulacije i evaluacija, a različiti koraci mogu da pristupaju tim podacima.

- Kako se odredjuje redosled izvrsavanja i vracanje na prethodni korak - Unutar Crew-a redosled može biti sekvencijalan ili koordinisan menadžerskim agentom

- CrewAI podržava asinhrono izvršavanje pojedinih zadataka, kao i koordinaciju više koraka kroz Flow

- CrewAI podržava mehanizme memorije i korišćenje prethodnih informacija tokom izvršavanja.
- Agent može koristiti dodatne alate za izvršavanje zadataka.
- CrewAI podržava mehanizme za proveru rezultata zadataka pre nego što se proslede dalje.
- CrewAI podržava prilagođene alate i integracije, uključujući MCP pristupe za povezivanje sa spoljnim funkcionalnostima.
- To znači da agent može koristiti alat koji poziva eksterni API.
- Ipak, CrewAI ti ne daje automatski kompletan generički registar svih udaljenih agenata prema njihovoj obrazovnoj ulozi.

- **Prednosti**:
- Jasno definisanje uloga i zadataka agenata.
- Jednostavno formiranje timova agenata.
- Podrška za sekvencijalnu i hijerarhijsku saradnju.
- Mogućnost upravljanja složenijim tokovima pomoću Flow-a.
- Podrška za alate, strukturisane izlaze i proveru rezultata.

- **Ogranicenja**:
- Za potrebe planiranog generičkog engine-a potrebno je dodatno razmotriti dinamičku registraciju agenata, njihovo pronalaženje prema sposobnostima i integraciju proizvoljnih eksternih agenata putem API-ja.

Microsoft Agent Framework

Dokumentacija: https://learn.microsoft.com/en-us/agent-framework/

GitHub: https://github.com/microsoft/agent-framework

Microsoft Agent Framework omogućava razvoj agenata i workflow-a koji povezuju agente i softverske funkcije kroz definisane tokove izvršavanja.

- Microsoft Agent Framework pruža mehanizme za razvoj agenata, njihovo povezivanje sa običnim softverskim funkcijama, kontrolu izvršavanja i integraciju sa drugim servisima.
- Microsoft Agent Framework omogućava da sve aktivnosti predstaviš kao koordinisan proces, a da pojedinačni koraci budu LLM agenti ili obične Python funkcije.

- **Agent** je komponenta koja koristi LLM da izvrši određeni zadatak.
- **Executor** – komponenta koja izvrsava posao. Ona prima podatke, izvršava posao i prosleđuje rezultat.
- **Workflow** – definisani tok izvrsavanja - predstavlja proces u kojem se izvršioci povezuju u određeni redosled
- **Orkestracija** – koordinacija vise agenata (odredjuje kako agenti međusobno sarađuju i kojim redosledom se izvršavaju)
- Microsoft Agent Framework ima više ugrađenih obrazaca orkestracije, uključujući sekvencijalno, paralelno, predaju kontrole, grupni razgovor i dinamičko koordinisanje pomoću menadžerskog agenta.

1. **Sequential** – sekvencijalna orkestracija

Agenti se izvršavaju jedan nakon drugog. Na primer, agent za kurikulum kreira plan, zatim agent za lekcije kreira sadržaj, pa agent za evaluaciju proverava rezultat.

2. **Concurrent** – paralelna orkestracija

Više agenata može istovremeno da obrađuje nezavisne zadatke. Na primer, agent za primere i agent za zadatke mogu raditi paralelno.

3. **Handoff** – predaja kontrole

Jedan agent može predati kontrolu drugom agentu kada proceni da je drugi pogodniji za nastavak zadatka. Na primer, nastavnik može prepustiti proveru odgovora specijalizovanom evaluatoru.

4. **Group Chat** – grupna komunikacija

Više agenata učestvuje u zajedničkoj konverzaciji. Ovo je zanimljivo za simulaciju učionice, gde agent nastavnik razgovara sa više agenata učenika.

5. **Magentic** – dinamička orkestracija

Menadžerski agent koordinira specijalizovane agente i odlučuje kako će raspodeliti posao. Korisno je za fleksibilne zadatke, ali može biti nepotrebno složeno za tvoju početnu implementaciju.

- Microsoft Agent Framework podržava korišćenje jednog workflow-a kao komponente drugog workflow-a.
- Framework omogućava čuvanje napretka workflow-a i kasniji nastavak izvršavanja.
- Sistem može da zaustavi izvršavanje i sačeka odluku čoveka.
- U Microsoft Agent Framework-u podaci se mogu prenositi porukama između izvršilaca.
- Redosled se određuje vezama i uslovima u workflow-u.
- Framework podržava događaje, metrike i praćenje toka izvršavanja.
- Microsoft Agent Framework podržava integracije sa različitim modelima, alatima i servisima, uključujući MCP.
- Ipak, to ne znači da automatski dobijaš kompletan sistem za registraciju proizvoljnih agenata na osnovu njihovih obrazovnih uloga.

Arhitektonske ideje iz naučne literature

EduPlanner – evaluacija i iterativna optimizacija

- Rad: https://arxiv.org/abs/2504.05370
- Fokus: generisanje, procena i unapređivanje nastavnih planova uz specijalizovane agente.
- Korisna ideja: **Evaluator → Optimizer → ponovna evaluacija**; evaluacija prema konkretnim kriterijumima, a ne samo jednom opštem utisku.
- Primena: agent za proveru kvaliteta lekcije vraća ocene i primedbe; engine pokreće odgovarajućeg agenta za doradu. Povratna petlja se može realizovati uslovnim vezama u LangGraph-u.
- Šta ne preuzimati odmah: kompletan Skill-Tree ili sve eksperimente iz rada; za početak je dovoljan jednostavan profil učenika i nekoliko kriterijuma kvaliteta.

SimClass – kontrola simulacije učionice

- Rad: https://doi.org/10.18653/v1/2025.naacl-long.520
- Fokus: simulacija interakcija nastavničkih agenata i agenata učenika.
- Korisna ideja: **Classroom Controller** koji određuje ko govori, kada se postavlja pitanje i kada se prelazi na narednu aktivnost.
- Primena: u trećoj fazi jedan kontroler koordinira nastavnika i nekoliko simuliranih učenika različitog predznanja; zapis interakcija prosleđuje se evaluaciji.
- Važno: rezultate simuliranih učenika ne treba predstavljati kao dokaz stvarnih obrazovnih ishoda.

AgentSchool – stanje znanja učenika

- Fokus iz prikupljenih beleški: modelovanje znanja i napretka simuliranih učenika.
- Korisna ideja: svaki učenik ima **eksplicitno stanje znanja** (npr. razumevanje klasifikacije, regresije, česte greške), koje se menja tokom simulacije.
- Primena: agent učenik dobija profil, odgovara u skladu sa njim, a nakon pitanja i povratnih informacija ažurira se njegovo stanje.
- Napomena: konkretan framework korišćen u radu nije potvrđen u dostupnim beleškama; pre akademskog citiranja treba proveriti originalni rad.

ChatDev – specijalizacija uloga i lanac zadataka

- GitHub: https://github.com/OpenBMB/ChatDev
- Fokus: koordinacija specijalizovanih agenata kroz faze i *Chat Chain*.
- Korisna ideja: složen proces podeliti na jasne zadatke, sa odgovornim agentima i definisanim rezultatima koji postaju ulaz narednih koraka.
- Primena: za svaku obrazovnu fazu definisati manje aktivnosti, očekivani izlaz i uslove za prelazak dalje.
- Napomena: razlikovati originalni istraživački sistem od kasnijih verzija repozitorijuma.

Generative Agents – memorija, refleksija i planiranje

- GitHub: https://github.com/joonspk-research/generative_agents
- Fokus: agenti koji koriste iskustva, refleksiju i planiranje.
- Korisna ideja: **Memory → Reflection → Planning**.

- Primena: nastavnik pamti prethodne greške simuliranog učenika i prilagođava naredno objašnjenje. U početnoj verziji dovoljni su istorija interakcija i strukturisani profil učenika; složenija dugoročna memorija može biti kasnije proširenje.

ADDIE i konstruktivno usklađivanje – obrazovni okvir

- ADDIE daje osnovu za razlaganje nastavnog procesa i iterativno unapređivanje njegovih faza.
- *Constructive Alignment* (Biggs) podseća da ciljevi učenja, nastavne aktivnosti i evaluacija moraju biti usklađeni.
- Primena: validatori proveravaju da li svaka lekcija i test imaju vezu sa odgovarajućim ciljem učenja; evaluacija može predložiti korekcije kurikuluma ili materijala.
- Ovo su **pedagoški okviri**, a ne softverski framework-i za AI agente.